Kehong Li

Date: 11/29/2025

SmartEats - AI Recipe & Dining Recommendation System

---

**Introduction**

In today's fast-paced world, almost everyone has the same basic question every day: "What should I eat?" It sounds like an easy question, but finding the answer is often difficult and takes a lot of time. This is because there is a big difference between the time we have and the information we need, people often spend too much time thinking about food instead of just enjoying it.

Most people get stuck in two situations. First, they might have a lot of food in the fridge, but they don't know what to make with it. For example, when people find some tomatoes and eggs in their kitchen, but they don't know which recipe can match them, or they might think "Oh, I cooked this food this way yesterday, and right now I want to try a different way". However, because people often run out of cooking ideas, they don't know how to use the ingredients they have which make good food sit around until it spoils and gets thrown away which creating unnecessary waste. Second, people might crave a specific type of food like spicy food or Italian cuisine but they don’t know how to make it by themselves, also they don’t know which restaurant nearby serves the best version of it.

Right now, there are already many websites and apps that try to help, but they usually only solve half of the problem. For example, websites like SuperCook

are great for finding recipes, but they require you to manually type in every single ingredient you have. On the other hand, websites like Jow can help create grocery lists, but they assume that they need to buy everything from scratch, they don't understand what people already have in their kitchen, so which means users have to switch between different apps to solve the simple problem, that is very inconvenient.

So, our goal is to combine these different ideas into one single platform called SmartEats. SmartEats uses Artificial Intelligence and Location-Based Services to bridge the gap between *"What do I have?"* and *"What do I want to eat?"*.

- If you want to cook, you can just take a picture of your fridge, and our system will tell you what to make.
- If you want to eat out, our system will show you the best restaurants nearby based on what you are craving.

By doing this, we hope to make dining decisions faster and smarter for everyone.

---

**Justification of Contribution**

The primary contribution of this project is the development of a unified decision-making framework that bridges the systemic disconnect between household inventory management and external dining services. Unlike existing siloed applications that force users to fragment their attention between manual recipe databases and restaurant finders, SmartEats justifies

its architectural design by integrating "Fridge-to-Recipe" and "Cuisine-to-Choice" workflows into a single, cohesive interface. This approach specifically addresses the limitations of the "zero-inventory" assumption in current grocery platforms by leveraging AI-driven image recognition to prioritize existing household stock, thereby significantly reducing the cognitive load of manual entry and mitigating food waste. By offering a simultaneous comparison of cooking, takeout, and dining out options, the system provides a comprehensive solution that optimizes user decisions based on real-time resource availability rather than platform constraints.

---

**Related Work**

Before building our project demo, we researched several existing websites and applications to understand the current landscape and identify viable options for our project. We analyzed three distinct categories of solutions: traditional recipe databases, ingredient-based search engines, and grocery integration platforms. Additionally, we examined location-based dining services to identify the missing link between home cooking and dining out.

The most traditional way for users to decide what to eat is by browsing large recipe databases like Allrecipes or Food Network, which employ a "search-first" model. Users typically decide what they want to eat, such as "lasagna" or "chicken curry," and then enter the dish name in the search bar then the system will return thousands of variations of that dish and ranked by user ratings or popularity.

However, this model relies on the user already knowing what they want to eat.

It's useless if a user is standing in front of an open refrigerator with scattered ingredients like eggs, bell peppers, and leftovers. Since these websites don't know what's in the user's fridge, the user might find a seemingly perfect recipe only to discover during cooking that key ingredients are missing, leading to frustration or a last-minute trip to the supermarket.

To address the "inventory blind spot" problem of traditional websites, platforms like SuperCook and MyFridgeFood have introduced the concept of "reverse recipe search." SuperCook's sidebar allows users to select ingredients from categories such as dairy, vegetables, and meat, and the recipe list is updated in real time. While the logic is sound good, but the user experience is somewhat inconvenient. This system relies entirely on manual input, users must scroll through long lists to find the corresponding ingredient button or enter the ingredient one by one. For users who need multiple ingredients, manually entering this data is both tedious and time consuming. Adding automated functions such as AI image recognition or voice input recognition would greatly increase convenience and improve the user experience.

Similar to SuperCook, MyFridgeFood also displays a row of checkboxes representing frequently used ingredients. However, MyFridgeFood's interface is often cluttered and confusing. More importantly, both SuperCook and MyFridgeFood are limited to "cooking silos," assuming users must cook for themselves. If users lack enough ingredients to cook a delicious meal, these platforms do not offer any alternatives, such as replenishing the missing ingredients or recommending nearby restaurants, which can leave users in a difficult situation.

Apps focused on grocery and meal planning, such as Jow and Yummly, are emerging platforms that combine meal planning with e-commerce. Jow asks users about their taste preferences and family size, then recommends a weekly menu. Once the menu is selected, Jow automatically adds all necessary ingredients to the user's online shopping cart.

The main flaw here lies in the "zero inventory" assumption. These apps are designed to sell groceries and operate on the premise that the user's kitchen is empty. For example, if a recipe calls for "salt," "olive oil," or "soy sauce". Jow will often add them to the cart as well, even if most households already own these condiments. This logic forces users to spend unnecessary money and time manually removing items from their carts, thus defeating the purpose of a "quick" solution.

On the other hand, restaurant search apps like Yelp and Google Maps excel at helping users find restaurants based on location, cuisine and price range. However, these platforms are disconnected from the user's household inventory, they don't know if the user has fresh salmon in their refrigerator that needs cooking. Therefore, instead of complementing home cooking they are compete with it. Users must open the recipe app to see if they know how to cook it, if not they must close the app and search for restaurants on Yelp. Therefore, there is currently no unified interface for comparing "cooking at home" and "dining out" side-by-side.

Through extensive research, we found that existing solutions operate independently either providing inventory management tools, recipe platforms, or restaurant apps, but consistently lacking an integrated decision making

system. Therefore, our project plans to integrate the following features to bridge this gap:

- AI-driven recognition: Introducing ingredient recognition to simplify user interaction and replace manual input.
- Smart inventory logic: Utilizing inventory-based reasoning to make precise shopping recommendations, purchasing only the missing ingredients to minimize waste.
- Unified platform: Integrating home cooking and dining out options into a unified interface to simplify the decision-making process.

---

**Problem and Task Definition**

Building on our previous discussions, SmartEats' primary goal is to address a major problem: the gap between the ingredients users have, the time they have, and the cooking decisions they need to make. Currently, existing apps are disconnected, so users have to constantly switch between inventory apps, recipe websites, and takeout apps to decide what to eat for dinner. To solve this problem, we designed SmartEats to handle two main tasks:

Task 1: Fridge-to-Recipe Mode (Smart Cooking)

The first task aims to help users make the most of the ingredients they have, avoiding waste or the hassle of choosing dishes.

- Input: We want this process to be extremely simple. Users do not need to enter each ingredient individually; simply use AI computer vision technology. The user just takes a picture of their refrigerator or

kitchen countertop.

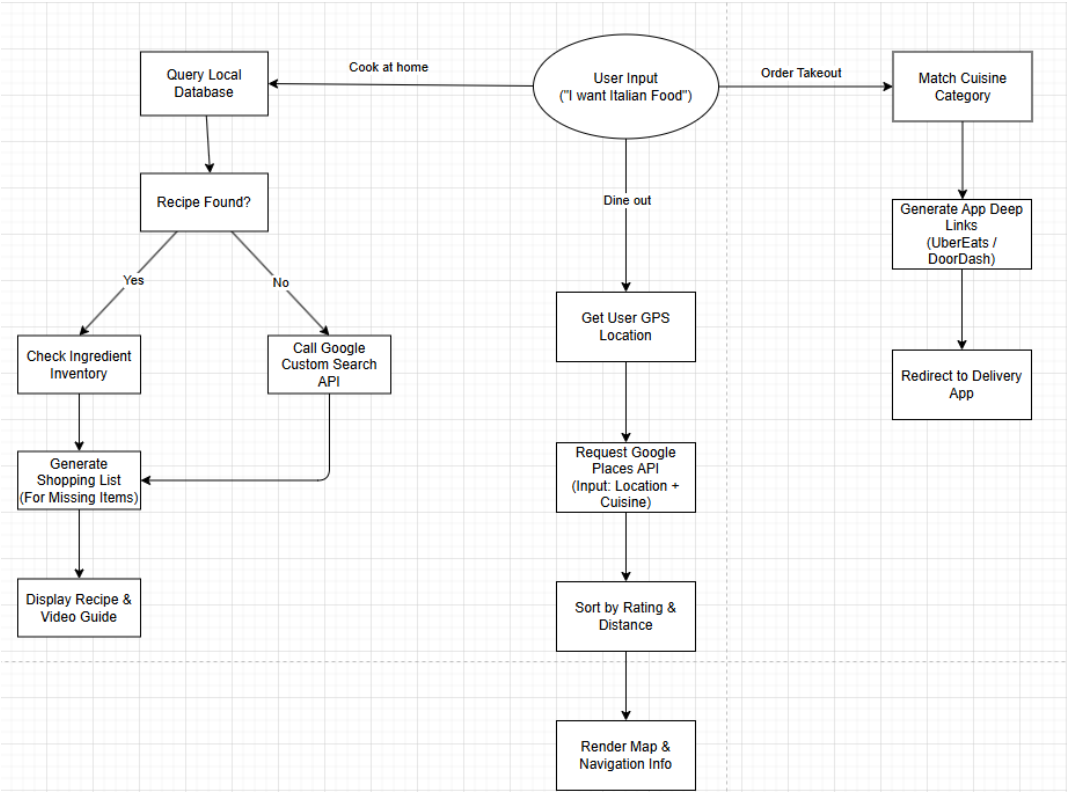
- Process: The system analyzes the user's photo, identifies the ingredients (and quantities), and finds recipes that match their existing ingredients.
- Output: The system provides the user with a list of dishes they can cook immediately.
- Video Guides: To make cooking easier for beginners, we not only provide text instructions but also AI-generated videos or short clips that allow users to intuitively follow the cooking steps.
- Smart Shopping: If the user has most of the ingredients but is missing one or two, the system will accurately calculate the missing ingredients and generate a precise shopping list, preventing them from buying items they already have.

Task 2: Cuisine-to-Choice Mode (Smart Dining)

The second task is suitable when the user first thinks about what they want to eat, like "I want Italian food" or "I want spicy food", rather than thinking about which ingredients they need. The system will provide the user with a more comprehensive selection than a standard restaurant list.

- Input: The user tells the system what they want to eat
- Process and Output: The system processes the user's needs and provides them with three clear solutions.
- Cooking at Home: Shows the user recipes that match their needs and provides a shopping list so they can buy the ingredients immediately.

- Ordering Takeout: Connects to apps like UberEats or DoorDash to recommend restaurants that deliver to the user's home.
- Dining Out: Uses location services (Google Maps) to show the highest-rated restaurants nearby, including distance and directions.




System Requirements

To ensure the application is professional and easy to use, it must meet the following three key requirements:

- Accuracy: The AI camera needs to be intelligent enough to correctly identify common foods from photos.
- Speed: When users enter the food that they want to eat, the system must immediately display three options—cook it yourself, order takeout, or eat out—without waiting.

- Unified Interface: All functions must be accessed through a unified interface. Users should be able to view maps or recipes without closing the application, it should truly be a "one-stop" platform for all their dining needs.

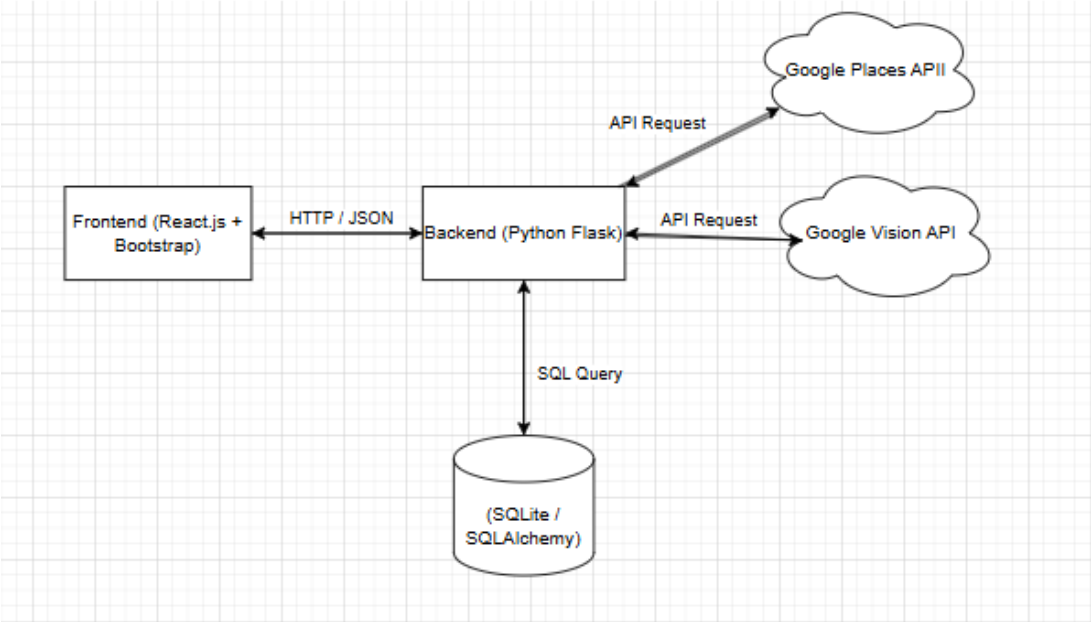
---

**Method and Implementation**

To build SmartEats, we designed a complete full-stack web system. We separated the project into three main parts: the User Interface (Frontend), the Server Logic (Backend), and the Data Storage (Database).

System Architecture

- Frontend: We used React.js and Bootstrap. This allows us to build a "Single Page Application," meaning the website is very fast and doesn't need to reload when switch between Cooking Mode and Dining Mode. Bootstrap ensures the site looks good on both phones and computers.
- Backend: The server is built with Python Flask. We chose Python because it is the best language for handling data logic and connecting to AI services. It acts as the bridge between the user and the database.
- Database: We used SQLite to store recipes and ingredients. To manage this data easily in our code, we used a tool called SQLAlchemy, which allows us to organize data without writing complex code.




Module 1: Fridge-to-Recipe Implementation

This module is designed to solve the "I have ingredients but don't know what to cook" problem.

- Step 1: AI Input. Instead of typing ingredients one by one, our design uses the Google Vision API. Users simply take a photo of their fridge, and the system automatically recognizes the ingredients ("Tomato," "Egg"…) and adds them to the virtual pantry.
- Step 2: The Matching Algorithm. We developed a Rule-based Matching algorithm in the backend. The system takes the list of ingredients the user has. It checks every recipe in the database to count how many required ingredients match what the user owns. If the match rate is 100%, it is a "Perfect Match."
- Step 3: Output and Guidance. The system shows a list of "Ready-to-Cook" recipes.
- Smart Shopping: If you are missing an ingredient (for example, you

have eggs but no tomato), the system calculates exactly what is missing and creates a precise shopping list.

- AI Video Support: To help beginners, the system is designed to integrate AI-generated instructional videos, allowing users to watch step-by-step cooking guides.

Module 2: Cuisine-to-Choice Implementation

This module handles the "I am hungry but don't know what to eat" problem. We implemented a "Three-Path Logic" based on user cravings.

- Path 1: Cook at Home. If the user wants to cook a specific cuisine ("Italian"), the system searches our local database. If no local recipes match, we use the Google Custom Search API to find relevant recipes from the web, generating a grocery list for immediate purchase.
- Path 2: Order Takeout. If the user prefers delivery, the system connects with food delivery platforms like UberEats or DoorDash. It recommends restaurants that serve the specific cuisine the user asked for.
- Path 3: Dine Out (Location-Based Services). If the user wants to go out, we use the Google Maps (Places) API. The backend takes the user's location and cuisine preference, sends it to Google, and returns a list of nearby highly rated restaurants with their distance, ratings, and navigation assistance.

Deployment

To make sure the project can run on any computer, we use Docker to package

the whole application. This ensures that the frontend, backend, and database work together consistently, making the software portable and easy to install.

---

**Experiments and Results**

This section details the actual development process of the SmartEats prototype and presents the functional results based on our current codebase.

Our development started from the backend, database, then frontend. We initiated the project by establishing the server infrastructure using Python Flask, which serves as the central API gateway. For data persistence, we utilized SQLAlchemy to construct a relational schema in models.py, defining two primary entities: the Recipe model to store cooking steps and cuisine types, and the RecipeIngredient model to map specific ingredients to dishes. To ensure the system was immediately testable upon deployment, we implemented an automated data seeding script (init_data) within the application entry point. This script checks for existing records and populates the SQLite database with initial seed data ("Tomato Egg Stir-Fry") whenever the container starts, preventing the application from launching with an empty dataset. We then modularized the business logic into a dedicated services layer to maintain code maintainability. Specifically, we implemented a set-theoretic intersection algorithm in services/recipes.py to handle ingredient matching and developed distinct modules for external API integrations—services/places.py for location-based restaurant queries and services/webrecipes.py for web-based recipe discovery.

On the frontend, we constructed a Single Page Application using React to

ensure a responsive user experience. We integrated Axios to manage asynchronous HTTP requests, allowing the frontend to communicate seamlessly with the Flask backend running on port 5001. To achieve a professional visual layout without extensive custom CSS, we employed the Bootstrap framework, utilizing its grid system to render recipe cards and restaurant lists side-by-side. Finally, to unify these disparate components, we implemented containerization using Docker. By defining specific Dockerfile configurations for both the Node.js frontend and Python backend, and orchestrating them via docker-compose.yml, we achieved a consistent deployment environment where both services could communicate within an isolated network, eliminating "works on my machine" dependency issues.

## Functional Verification and Performance Evaluation

We conducted comprehensive functional testing to verify the system's core capabilities across different user scenarios. In the "Cook at Home" scenario, we validated the local matching logic by inputting specific ingredients ("Tomato" and "Egg"). The backend logic in recipes.py successfully queried the SQLite database and returned "Tomato Egg Stir-Fry" with a 100% match score. Furthermore, we tested the smart shopping list feature by removing an ingredient; the system correctly recalculated the match ratio and identified the missing item in the response payload. We also verified the fallback mechanism: when random ingredients ("Chicken") were entered that did not exist in the local database, the system automatically triggered the discover_recipes_from_web function, successfully fetching and displaying relevant recipe links from the Google Custom Search API, proving the system's robustness against limited local data.

For the "Dine Out" functionality, we tested the integration with Location-Based Services. Upon selecting the "Italian" cuisine preference in the frontend, the backend successfully transmitted the request to the Google Places API via services/places.py. The system accurately rendered a list of real-world Italian restaurants located near the test coordinates (Mansfield, CT), complete with star ratings and addresses.

---

**Conclusion**

The primary goal of this graduation project is to address the systemic disconnect between household inventory management and dining decisions. By designing and implementing SmartEats, we successfully created a unified intelligent platform that bridges the gap between "what users own" and "what they want to eat." We built a complete full-stack system using React and Flask, with an SQLite database as the backend and containerized deployment using Docker, demonstrating that simplifying the dining decision-making process into a unified interface is entirely feasible.

Our functional testing confirms that the system effectively handles its two core tasks. The "refrigerator to recipe" mode successfully leverages local database matching and web search as alternatives to transform existing ingredients into actionable cooking plans; while the "cuisine to choice" mode utilizes location-based services to provide instant and context-appropriate dining options. SmartEats demonstrates that by integrating inventory logic with external discovery APIs, we can transform everyday stressors into a seamless, data-driven experience, ultimately helping users make smarter, faster, and more

sustainable food choices.

Future work will includes expanding the dataset, improving ingredient recognition accuracy, integrating nutrition analysis, and developing a mobile-native version with offline support.

---

**Broader Impact**

Environmental Sustainability: Reducing Food Waste is one of the most pressing problems facing modern households is food waste, often caused by ingredients spoiling before consumption. Traditional shopping habits often lead to overbuying, while a lack of culinary inspiration leaves existing stock unused. SmartEats addresses this problem directly with its "stock-first" design philosophy. By prioritizing a "refrigerator-to-recipe" model, the system encourages users to create meals based on what they already have, rather than what they need to buy. This shift in behavior helps prevent edible food from ending up in landfills, thus reducing the overall carbon footprint associated with food production and waste disposal

Public Health and Lifestyle In today's decision-making fatigue, many people opt for fast food or processed foods simply because the cognitive burden of planning to cook at home is too heavy. SmartEats eliminates these barriers by simplifying the process—just take a picture. By providing instant recipe matching and lowering the cooking threshold, the platform promotes a healthier lifestyle. It allows users to choose homemade meals, which often offer better control over ingredients, portion sizes, and nutritional value compared to restaurant dining.

Economic Benefits: From a financial perspective, SmartEats offers tangible economic benefits to households, especially given rising food prices. Unlike promotional grocery apps that encourage bulk purchases, our system's smart shopping list algorithm accurately calculates the difference between the ingredients needed for a recipe and existing inventory. This precision prevents users from making duplicate purchases—such as buying a second can of spices or a second bag of flour—thus optimizing household spending and ensuring that grocery shopping is both efficient and necessary.

---

**References**


1. **React Documentation.** *React: A JavaScript library for building user interfaces.* Meta Platforms, Inc. Available at: https://react.dev/
2. **Flask Documentation.** *Flask: A lightweight WSGI web application framework.* Pallets Projects. Available at: https://flask.palletsprojects.com/
3. **Google Cloud.** *Cloud Vision API: Derive insights from your images in the cloud.* Google Cloud Platform. Available at: https://cloud.google.com/vision
4. **Google Maps Platform.** *Places API: Search for places and businesses.* Google Developers. Available at: https://developers.google.com/maps/documentation/places/web-service
5. **SQLAlchemy.** *The Python SQL Toolkit and Object Relational Mapper.*

SQLAlchemy.org. Available at: https://www.sqlalchemy.org/

6. **Min, W., Jiang, S., Liu, L., Rui, Y., & Jain, R.** (2019). "A Survey on Food Computing." *ACM Computing Surveys (CSUR)*, 52(5), 1-36.

7. **Docker Documentation.** *Containerize an application.* Docker Inc. Available at: https://docs.docker.com/

8. **Bootstrap Documentation.** *Bootstrap: The most popular HTML, CSS, and JS library in the world.* Available at: https://getbootstrap.com/